ASSIGNMENT – 4

1. The inner query solves first then outer query.

   The subquery calculates the average salary of all employees. The outer query then compares each employee's salary with the value return by inner query and returns only those earning more.

```
postgres=# SELECT name, salary
postgres-# FROM employees
postgres-# WHERE salary > (SELECT AVG(salary) FROM employees);
 name  |  salary
-------+----------
 John  | 80000.00
 Riya  | 75000.00
 Kabir | 95000.00
 Arjun | 85000.00
(4 rows)
```

2. ORDER BY salary DESC sorts salaries from highest to lowest. LIMIT 5 ensures only the first five rows (top earners) are returned. Sorting determines which rows qualify as "top."

```
postgres=# SELECT name, salary
postgres-# FROM employees
postgres-# ORDER BY salary DESC
postgres-# LIMIT 5;
 name  |  salary
-------+----------
 Kabir | 95000.00
 Arjun | 85000.00
 John  | 80000.00
 Riya  | 75000.00
 Priya | 72000.00
(5 rows)
```

3. **Aggregate functions:** Work on multiple rows and return one result (COUNT, AVG, SUM).

   Scalar functions: Work on a single value and return one value (UPPER, ROUND, LENGTH).

```
postgres=# SELECT
postgres-#     COUNT(*) AS total_employees,
postgres-#     AVG(salary) AS avg_salary,
postgres-#     MIN(salary) AS min_salary,
postgres-#     MAX(salary) AS max_salary
postgres-# FROM employees;
 total_employees |     avg_salary     | min_salary | max_salary
-----------------+--------------------+------------+------------
               8 | 73375.000000000000 |   50000.00 |   95000.00
(1 row)
```

4. GROUP BY region groups rows by region.

   HAVING filters groups after aggregation (WHERE cannot filter aggregated values).

```
postgres-# FROM sales
postgres-# GROUP BY region
postgres-# HAVING SUM(amount) > 50000;
 region | total_sales
--------+-------------
 East   |    60000.00
 West   |    55000.00
 North  |    55000.00
(3 rows)
```

5. Without DISTINCT, COUNT would include duplicate job roles. DISTINCT ensures each role is counted once.

```
postgres=# SELECT COUNT(DISTINCT job_role) AS unique_roles
postgres-# FROM employees;
 unique_roles
--------------
            5
(1 row)
```

6.

```
postgres=# SELECT *
postgres-# FROM students
postgres-# WHERE marks >= 60 AND marks <= 80;
 student_id |  name  | marks
------------+--------+-------
          1 | Rahul  |    75
          2 | Sneha  |    62
          6 | Anjali |    68
(3 rows)


postgres=# SELECT *
postgres-# FROM students
postgres-# WHERE marks BETWEEN 60 AND 80;
 student_id |  name  | marks
------------+--------+-------
          1 | Rahul  |    75
          2 | Sneha  |    62
          6 | Anjali |    68
(3 rows)
```

7. We must use IS NULL or IS NOT NULL. Using = NULL won't work because NULL represents an unknown value, not a literal.

```
postgres=# SELECT *
postgres-# FROM employees
postgres-# WHERE commission IS NULL;
 emp_id | name  | department | job_role  |  salary  | commission
--------+-------+------------+-----------+----------+------------
      1 | Amit  | IT         | Developer | 70000.00 |
      3 | John  | IT         | Developer | 80000.00 |
      5 | Kabir | IT         | Team Lead | 95000.00 |
      6 | Neha  | HR         | Executive | 50000.00 |
      8 | Priya | IT         | Developer | 72000.00 |
(5 rows)
```

8. SQL supports arithmetic operations directly in queries. Here, each salary is multiplied by 1.10 to add 10%.

```
postgres=# UPDATE employees
postgres-# SET salary = salary * 1.10
postgres-# WHERE department = 'IT';
UPDATE 4
postgres=# select * from employees;
 emp_id | name  | department | job_role  |  salary   | commission
--------+-------+------------+-----------+-----------+------------
      2 | Sara  | HR         | Manager   |  60000.00 |    5000.00
      4 | Riya  | Finance    | Analyst   |  75000.00 |    3000.00
      6 | Neha  | HR         | Executive |  50000.00 |
      7 | Arjun | Finance    | Manager   |  85000.00 |    7000.00
      1 | Amit  | IT         | Developer |  77000.00 |
      3 | John  | IT         | Developer |  88000.00 |
      5 | Kabir | IT         | Team Lead | 104500.00 |
      8 | Priya | IT         | Developer |  79200.00 |
(8 rows)
```

9. Always run a SELECT with the same condition first.

```
postgres=# DELETE FROM students
postgres-# WHERE marks < 40;
DELETE 1
postgres=# select * from students;
 student_id |  name  | marks
------------+--------+-------
          1 | Rahul  |    75
          2 | Sneha  |    62
          3 | Karan  |    45
          4 | Meera  |    82
          6 | Anjali |    68
          7 | Rohit  |    55
(6 rows)


postgres=# SELECT * FROM students WHERE marks < 40;
 student_id | name | marks
------------+------+-------
(0 rows)
```

10. The subquery calculates the average salary for the department of the current employee (e.department). The outer query compares each employee's salary with their department's average and returns only those above it.

```
postgres=# SELECT name, salary, department
postgres-# FROM employees e
postgres-# WHERE salary >
postgres-# (
postgres(#     SELECT AVG(salary)
postgres(#     FROM employees
postgres(#     WHERE department = e.department
postgres(# );
 name  |  salary   | department
-------+-----------+------------
 Sara  |  60000.00 | HR
 Arjun |  85000.00 | Finance
 John  |  88000.00 | IT
 Kabir | 104500.00 | IT
(4 rows)
```